

# Universidad Internacional del Ecuador



**Ingeniería en software**

**Nombre:**

**Daniela Paillacho**

**Asignatura:**

**PROGRAMACION ORIENTADA A OBJETOS**

**Fecha de envió: 23/02/26**

**Fecha de entrega:01/03/26**

**Quito-Ecuador**

**2025-2026**

## Selección de la Aplicación Web

### Aplicación seleccionada:

Sistema Web de Gestión de Libros Electrónicos

### Justificación

Se seleccionó esta aplicación porque:

Permite aplicar Programación Orientada a Objetos (POO) mediante clases como:

- ✓ Libro
- ✓ Usuario
- ✓ Prestamo
- ✓ GestorLibros

### Es adecuada para implementar concurrencia, ya que múltiples usuarios pueden:

- ✓ Registrarse simultáneamente
- ✓ Consultar libros al mismo tiempo
- ✓ Solicitar préstamos concurrentes

### Se adapta bien a tecnologías como:

- ✓ Flask (Framework web)
- ✓ SQLite (Base de datos)
- ✓ Permite aplicar pruebas unitarias e integración fácilmente.

## Diseño y Desarrollo

### Arquitectura de la Aplicación

#### Arquitectura tipo MVC:

- ✓ Modelo (Model) → Clases POO
- ✓ Vista (View) → Plantillas HTML
- ✓ Controlador (Controller) → Rutas Flask

## Clases Principales (POO)

```
1  class Libro:
2      def __init__(self, titulo, autor, isbn, formato, disponible=True):
3          self.__titulo = titulo
4          self.__autor = autor
5          self.__isbn = isbn
6          self.__formato = formato
7          self.__disponible = disponible
8
9      def prestar(self):
10         if self.__disponible:
11             self.__disponible = False
12             return True
13         return False
14
15     def devolver(self):
16         self.__disponible = True
```

### Clase: Usuario

```
18     class Usuario:
19         def __init__(self, nombre, correo):
20             self.__nombre = nombre
21             self.__correo = correo
22             self.__historial = []
23
24         def agregar_prestamo(self, prestamo):
25             self.__historial.append(prestamo)
```

### Clase: Préstamo

```
25     class Prestamo:
26         def __init__(self, libro, usuario, fecha_inicio, fecha_fin):
27             self.libro = libro
28             self.usuario = usuario
29             self.fecha_inicio = fecha_inicio
30             self.fecha_fin = fecha_fin
31             self.estado = "Activo"
32
```

## Implementación de Concurrency

En Flask, la concurrencia puede manejarse mediante:

- ✓ Uso de threading
- ✓ Configuración del servidor en modo multihilo
- ✓ Manejo seguro de la base de datos

```
from threading import Lock

lock = Lock()

def prestar_libro(libro):
    with lock:
        return libro.prestar()
```

✚ Esto evita que dos usuarios presten el mismo libro al mismo tiempo.

## Pruebas y Optimización

Se utilizó **unittest** para probar clases individuales.

```
import unittest

class TestLibro(unittest.TestCase):
    def test_prestar_libro(self):
        libro = Libro("Python", "Autor X", "123", "PDF")
        self.assertTrue(libro.prestar())
        self.assertFalse(libro.prestar())
```

## Pruebas de Integración

Se probaron:

- ✓ Registro de usuario + préstamo
- ✓ Registro de libro + búsqueda
- ✓ Autenticación + acceso restringido

Pruebas de Aceptación

| Caso                               | Resultado esperado | Resultado obtenido |
|------------------------------------|--------------------|--------------------|
| Usuario registra libro             | Libro guardado     | Correcto           |
| Usuario presta libro disponible    | Estado cambia      | Correcto           |
| Usuario presta libro no disponible | Error              | Correcto           |
| Búsqueda por título                | Lista filtrada     | Correcto           |

## Diagrama de Clases



## Optimización Implementada

- ✓ Validación de datos antes de guardar
- ✓ Manejo de errores con try/except
- ✓ Uso de bloqueo (Lock) para concurrencia
- ✓ Separación de capas (Modelo – Vista – Controlador)

Github: <https://github.com/Anahy2006/-Sistema-de-Gesti-n-de-Libros-electr-nicos-/tree/main>